

Análisis de Metadatos II: Unidad 2- Parte 1

Lenguaje de programación Python

Jair Pallares Castellon, MSc.



Introducción a lectura y manipulación de archivos

Métodos

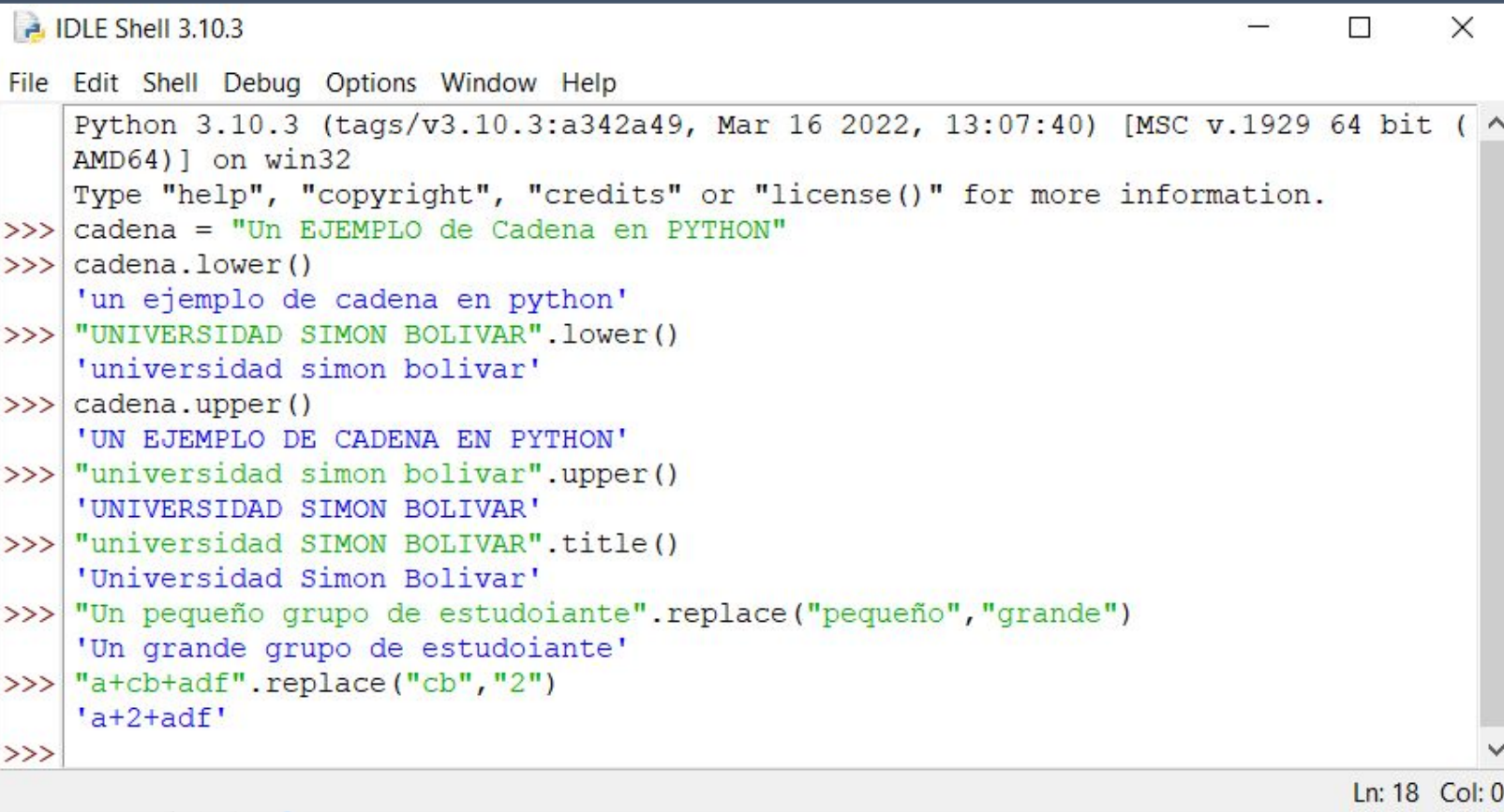
Los datos de ciertos tipos permiten invocar funciones especiales, a esto se le conoce como métodos. La forma en que podemos usarlos es la siguiente:

argumento1.método(argumento2, argumento3...)

Este argumento te dice que, para un método, el primer argumento es muy especial. Es como el sujeto de una frase de la que el método es el verbo.

1. Método **lower()**: Transforma el argumento en minúscula.
2. Método **upper()**: Transforma el argumento en mayúscula.
3. Método **title()**: Transforma la primera letra de cada palabra en mayúscula.
4. Método **replace()**: recibe como argumento dos cadenas: un patrón y un reemplazo.

Lectura y manipulación de archivos



```
IDLE Shell 3.10.3
File Edit Shell Debug Options Window Help
Python 3.10.3 (tags/v3.10.3:a342a49, Mar 16 2022, 13:07:40) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> cadena = "Un EJEMPLO de Cadena en PYTHON"
>>> cadena.lower()
'un ejemplo de cadena en python'
>>> "UNIVERSIDAD SIMON BOLIVAR".lower()
'universidad simon bolivar'
>>> cadena.upper()
'UN EJEMPLO DE CADENA EN PYTHON'
>>> "universidad simon bolivar".upper()
'UNIVERSIDAD SIMON BOLIVAR'
>>> "universidad SIMON BOLIVAR".title()
'Universidad Simon Bolivar'
>>> "Un pequeño grupo de estuoiante".replace("pequeño", "grande")
'Un grande grupo de estuoiante'
>>> "a+cb+adf".replace("cb", "2")
'a+2+adf'
Ln: 18 Col: 0
```

Un métodos más complejo: format

Es un método que permite al usuario ver los resultados de forma más atractiva. El método **format** de las cadenas nos proporciona una herramienta fundamental, aunque cuesta un poco dominarlo.

Primero debemos aprender a interpolar valores en una cadena, lo que implica sustituir una marca especial por el valor de una expresión.

El método **format** presenta algunas ventajas:

- Se pueden reemplazar números, mientras que con **replace** solo se reemplazan cadenas.
- Podemos interpolar más de un valor con una sola llamada.
- Cada marca de la forma **{ n }**, donde n es el número entero, se sustituye por el argumento de **format**: la marca **{0}** se asocia a sustituir el primer argumento, **{1}** el segundo argumento, etc.

Lectura y manipulación de archivos

IDLE Shell 3.10.3

File Edit Shell Debug Options Window Help

```
Python 3.10.3 (tags/v3.10.3:a342a49, Mar 16 2022, 13:07:40) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> "El número {0} ha sido interpolado.".format(1.45)
'El número 1.45 ha sido interpolado.'
>>> "El número {0} ha sido interpolado.".replace("{0}", "1.45")
'El número 1.45 ha sido interpolado.'
>>> "Los números {0} y {1} han sido interpolados."format(1.45, 9.67)
SyntaxError: invalid syntax
>>> "Los números {0} y {1} han sido interpolados.".format(1.45, 9.67)
'Los números 1.45 y 9.67 han sido interpolados.'
>>> "Los números {1} y {0} han sido interpolados.".format(1.45, 9.67)
'Los números 9.67 y 1.45 han sido interpolados.'
>>> |
```

Ln: 13 Col: 0

Lectura y manipulación de archivos

Marcas de formato

Permiten controlar con precisión el modo en el que se muestran los datos. La forma general de una marca de formato es esta:

{campo!marca de conversión:formato}

El **campo** es el número que identifica el número de argumento que se desea interpolar en la cadena. Para la parte de **:formato** su forma general es:

[[relleno]alineamiento][signo][#][0][ancho][.precisión][código de tipo]

Cada elemento entre corchetes es opcional.

- **relleno**: Carácter con el que rellenar los espacios que requiere un alineamiento.
- **alineamiento**: Carácter < para alinear a la izquierda en el espacio disponible; carácter > para alinear a la derecha en el espacio disponible; carácter ^ para centrar en el espacio disponible.

Lectura y manipulación de archivos

Marcas de formato

[[relleno]alineamiento][signo][#][0][ancho][.precisión][código de tipo]

Cada elemento entre corchetes es opcional.

- **#**: Si aparece, los enteros que se muestran en binario, octal o hexadecimal irán precedidos por **0b**, **0o** o **0x**.
- **0**: Si aparece, se usa el carácter o para sustituir los espacios en blanco.
- **ancho**: Es un número entero que indica cuántos caracteres queremos que ocupe el valor representado.
- **.precisión**: número de decimales con que queremos representar un número flotante.
- **código de tipo**: carácter que indica el tipo de representación que se desea. Es diferente según el tipo de datos del valor.

Lectura y manipulación de archivos

```
IDLE Shell 3.10.3
File Edit Shell Debug Options Window Help
Python 3.10.3 (tags/v3.10.3:a342a49, Mar 16 2022, 13:07:40) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> # EJEMPLOS
>>> "El {0} formateado.".format(123)
'El 123 formateado.'
>>> # 10 espacios
>>> "El {0:>10} formateado.".format(123)
'El          123 formateado.'
>>> "El {0:^10} formateado.".format(123)
'El      123      formateado.'
>>> "El {0:<10} formateado.".format(123)
'El 123          formateado.'
>>> # El 0 sustituye los espacios en blanco por un 0
>>> "El {0:010} formateado.".format(123)
'El 0000000123 formateado.'
>>> # Ejemplo de un carácter de signo
>>> "El {0:+} formateado.".format(123)
'El +123 formateado.'
>>> "El {0:-} formateado.".format(123)
'El 123 formateado.'
>>> "El {0: } formateado.".format(123)
'El 123 formateado.'
>>> |
```

Ln: 23 Col: 0

Posibles valores de un código de tipo:

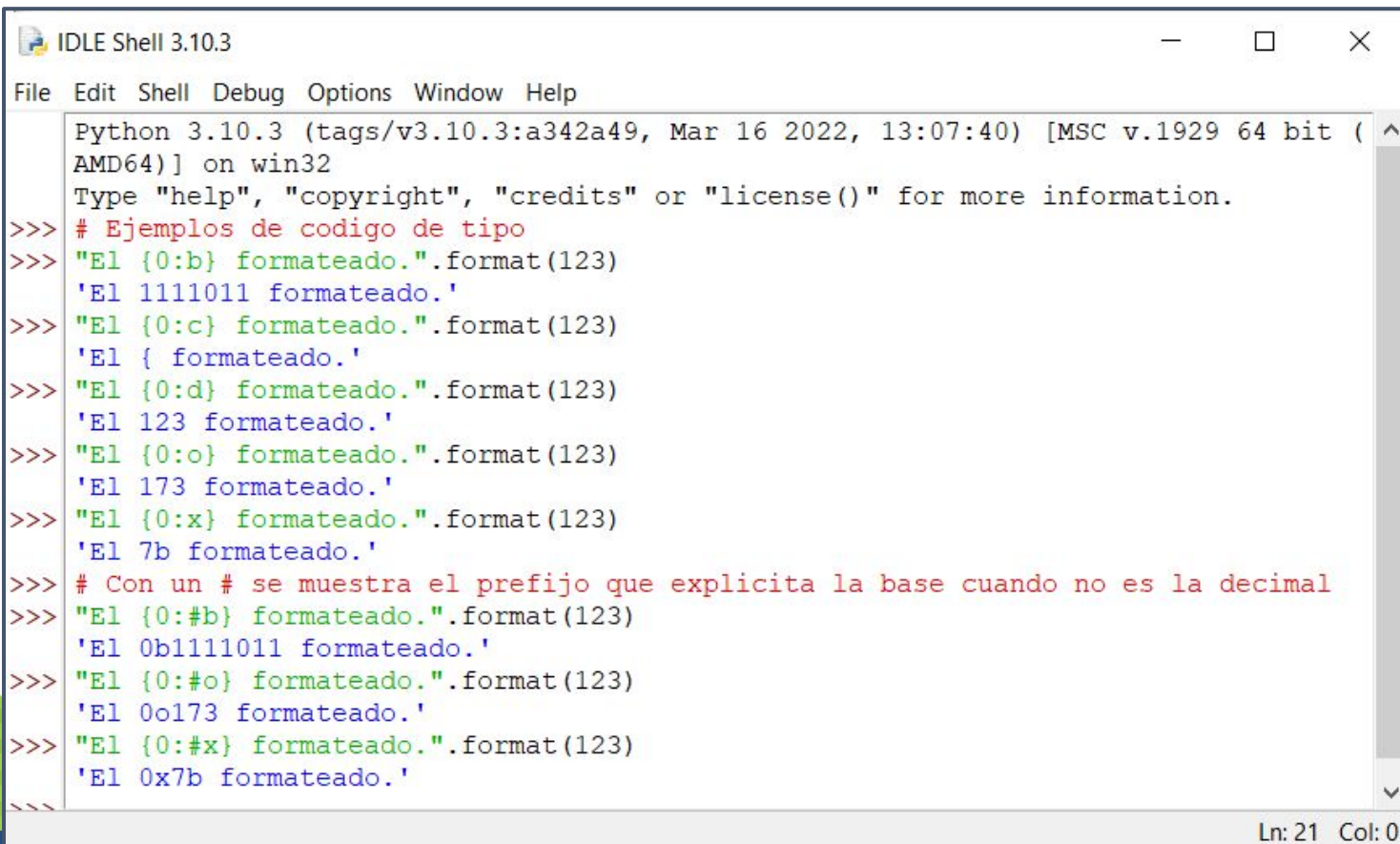
- Números enteros:

- carácter **b**: en binario.
- carácter **c**: como carácter Unicode.
- carácter **d**: en base diez.
- carácter **o**: en octal.
- carácter **x**: en hexadecimal.
- carácter **n**: punto decimal como se muestra como coma (español).

- Números flotantes:

- carácter **e**: notación exponente.
- carácter **f**: notación de punto fijo.
- carácter **g**: notación en formato general.
- carácter **n**: número adaptable a la cultura local.
- carácter **%**: muestra el número multiplicado por 100..

Lectura y manipulación de archivos



```
IDLE Shell 3.10.3
File Edit Shell Debug Options Window Help
Python 3.10.3 (tags/v3.10.3:a342a49, Mar 16 2022, 13:07:40) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> # Ejemplos de codigo de tipo
>>> "El {0:b} formateado.".format(123)
'El 1111011 formateado.'
>>> "El {0:c} formateado.".format(123)
'El { formateado.'
>>> "El {0:d} formateado.".format(123)
'El 123 formateado.'
>>> "El {0:o} formateado.".format(123)
'El 173 formateado.'
>>> "El {0:x} formateado.".format(123)
'El 7b formateado.'
>>> # Con un # se muestra el prefijo que explicita la base cuando no es la decimal
>>> "El {0:#b} formateado.".format(123)
'El 0b1111011 formateado.'
>>> "El {0:#o} formateado.".format(123)
'El 0o173 formateado.'
>>> "El {0:#x} formateado.".format(123)
'El 0x7b formateado.'
>>>
```

Ln: 21 Col: 0

Lectura y manipulación de archivos

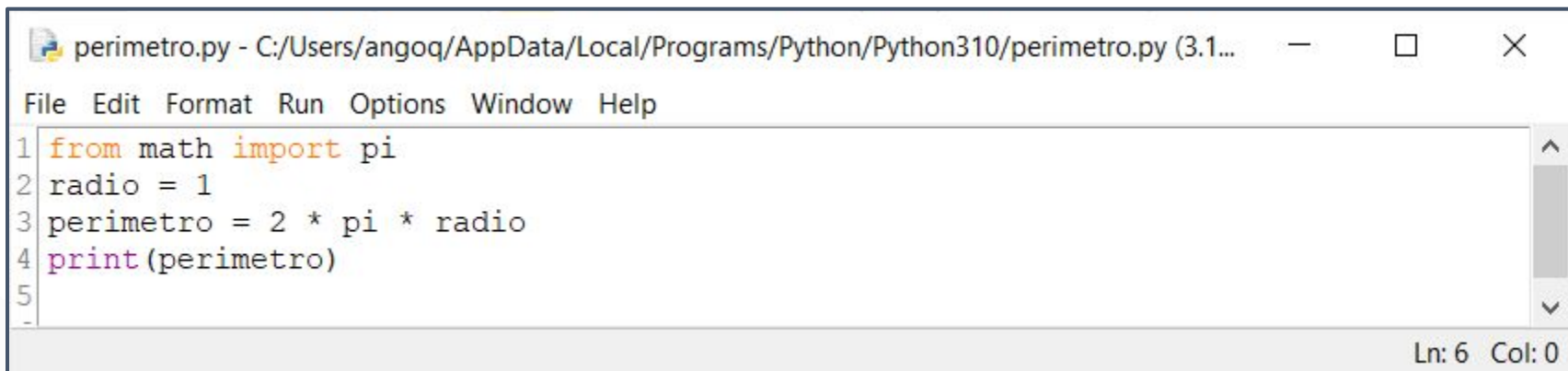
Nuestro primer programa

Hasta ahora hemos trabajado directamente en la consola de Python, en donde realizamos las operaciones y observábamos los resultados en el mismo lugar.

Ahora crearemos un **fichero** en donde escribiremos nuestro programa. Existen diferentes entornos de trabajo para Python (más adelante puedes escoger trabajar en el que desees), pero nosotros continuaremos trabajando con IDLE.

1. Debes abrir **Python IDLE** como usualmente lo hemos venido haciendo.
2. Selecciona **File** y luego **New File**.
3. Luego en la nueva ventana selecciona **Save as** y le damos un nombre, por ejemplo: **perimetro.py**.
4. Escribe un programa (**ver ejemplo** en la siguiente presentación)
5. Una vez escrito seleccionas **Run** en el fichero que has creado.
6. y finalmente tendrás tu resultado.

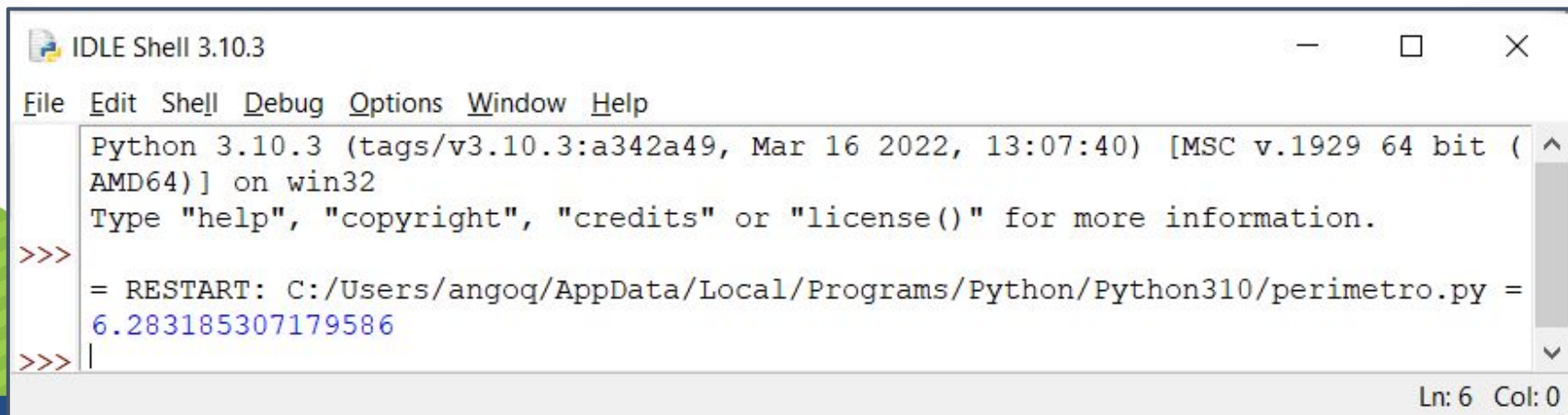
Lectura y manipulación de archivos



The screenshot shows a window titled 'perimetro.py - C:/Users/angoq/AppData/Local/Programs/Python/Python310/perimetro.py (3.1...'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following Python code:

```
1 from math import pi
2 radio = 1
3 perimetro = 2 * pi * radio
4 print(perimetro)
5
```

The status bar at the bottom right indicates 'Ln: 6 Col: 0'.



The screenshot shows the 'IDLE Shell 3.10.3' window. The menu bar includes File, Edit, Shell, Debug, Options, Window, and Help. The shell displays the following output:

```
Python 3.10.3 (tags/v3.10.3:a342a49, Mar 16 2022, 13:07:40) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/angoq/AppData/Local/Programs/Python/Python310/perimetro.py =
6.283185307179586
>>> |
```

The status bar at the bottom right indicates 'Ln: 6 Col: 0'.

Lectura y manipulación de archivos

Entrada/salida

El fichero descrito anteriormente tiene un error y es que al cambiar el programa debes ir al fichero y modificarlos. Por ejemplo: anteriormente calculamos el perímetro con un valor de radio 1, pero si ahora debo hacer el mismo cálculo con radio 5, debo ir al fichero **perímetro.py**.

Haremos que el programa nos pida por ejemplo el valor del radio sin necesidad de ir al fichero a modificarlo.

Para esto usamos la función **input()** (entrada). Si deseas que el radio de nuestro ejemplo sea un valor flotante debes usar una función **float()**. Recuerda que también existe la función **int()** (entero) que te da un entero en lugar de un flotante (valor con punto).

Lectura y manipulación de archivos

```
perimetro.py - C:\Users\angoq\AppData\Local\Programs\Python\Python310\perimetro.py (3.10....
```

File Edit Format Run Options Window Help

```
1 from math import pi
2
3 cadena_leida = input("Ingresa el radio: ")
4
5 radio = float(cadena_leida)
6
7 perimetro = 2 * pi * radio
8
9 print(perimetro)
10
```

```
IDLE Shell 3.10.3
```

File Edit Shell Debug Options Window Help

```
Python 3.10.3 (tags/v3.10.3:a342a49, Mar 16 2022, 13:07:40) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: C:\Users\angoq\AppData\Local\Programs\Python\Python310\perimetro.py =
Ingresa el radio: 5
31.41592653589793

>>>
= RESTART: C:\Users\angoq\AppData\Local\Programs\Python\Python310\perimetro.py =
Ingresa el radio: 8
50.26548245743669

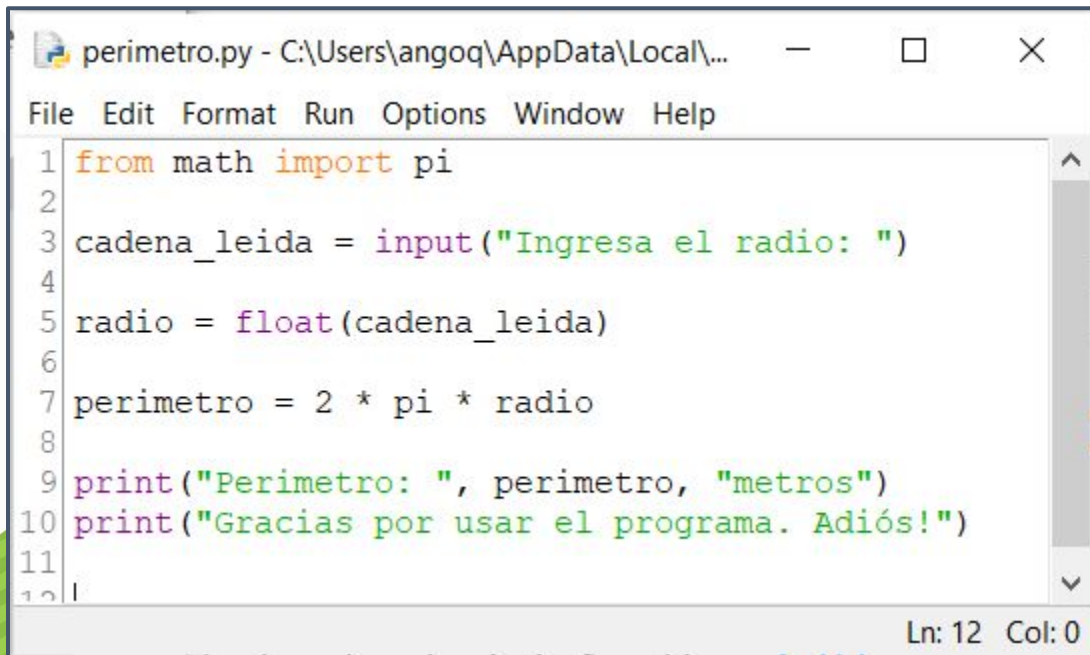
>>>
```

Ln: 8 Col: 0

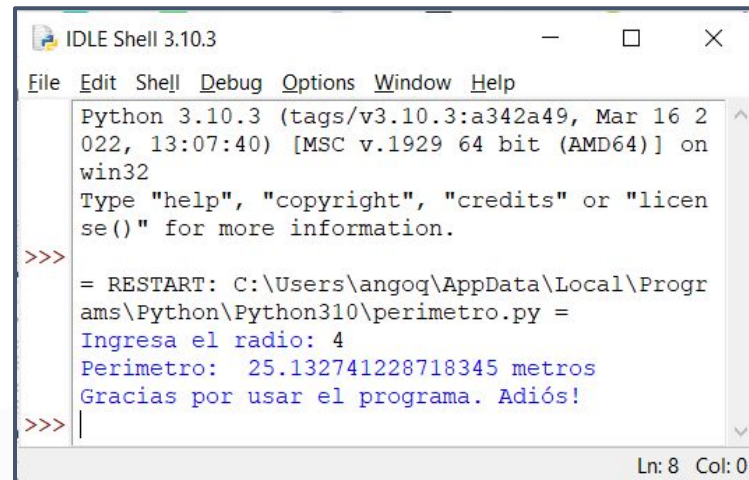
Lectura y manipulación de archivos

Más sobre la función print

Las cadenas pueden usarse para mostrar texto a través de la función `print()`.



```
perimetro.py - C:\Users\angoq\AppData\Local\...  
File Edit Format Run Options Window Help  
1 from math import pi  
2  
3 cadena_leida = input("Ingresa el radio: ")  
4  
5 radio = float(cadena_leida)  
6  
7 perimetro = 2 * pi * radio  
8  
9 print("Perimetro: ", perimetro, "metros")  
10 print("Gracias por usar el programa. Adiós!")  
11  
12 |  
Ln: 12 Col: 0
```



```
IDLE Shell 3.10.3  
File Edit Shell Debug Options Window Help  
Python 3.10.3 (tags/v3.10.3:a342a49, Mar 16 2022, 13:07:40) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
= RESTART: C:\Users\angoq\AppData\Local\Programs\Python\Python310\perimetro.py =  
Ingresa el radio: 4  
Perimetro: 25.132741228718345 metros  
Gracias por usar el programa. Adiós!  
>>> |  
Ln: 8 Col: 0
```